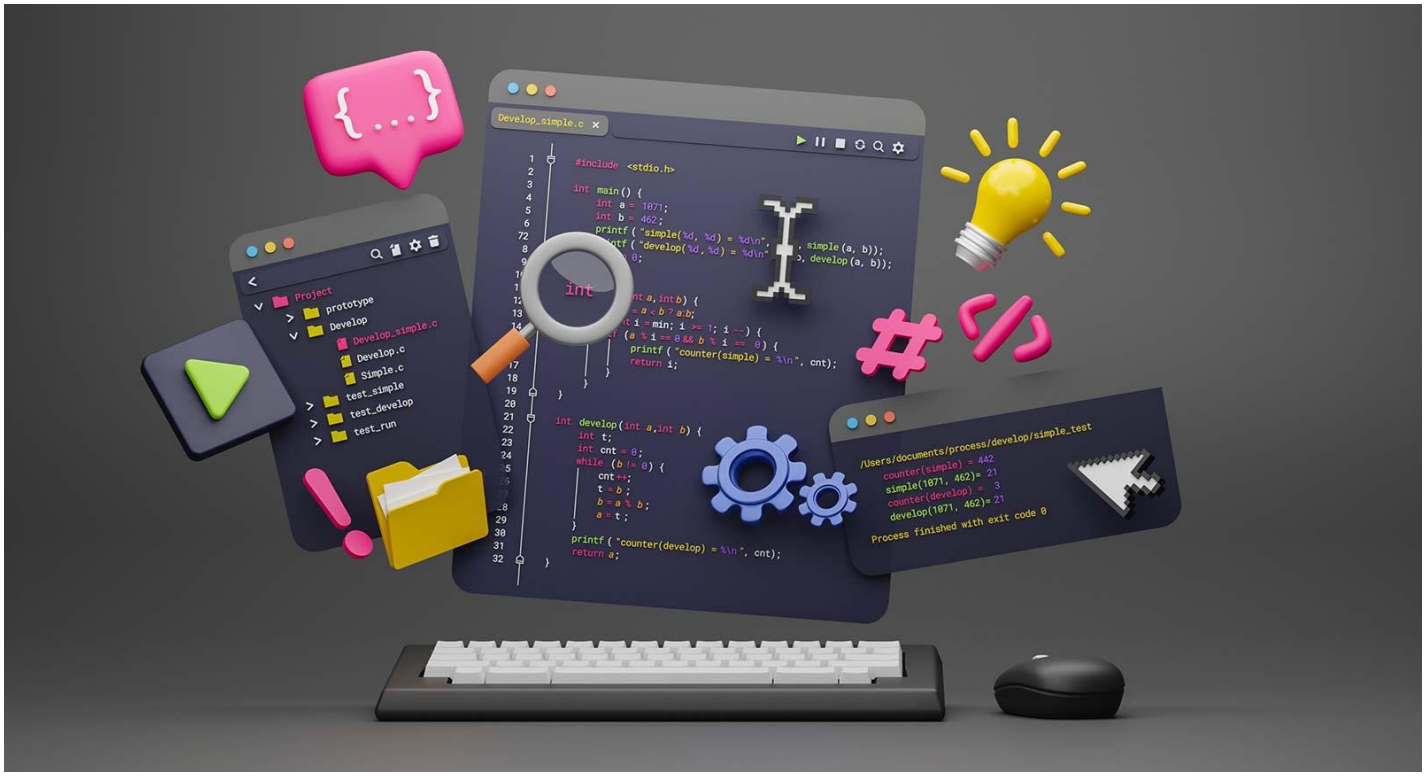
Université Claude Bernard Lyon 1

MIF01 : Gestion de Projet et Génie logiciel

Microblogging



ENCADRANT : M. MATTHIEU MOY
AMINE NEDJAR
HAMID HAKI

Table des matières :

1 Présentation du projet : 1
1.1 Fonctionnement de l'application : 1
2 Design Patterns : 1
2.1 Patron d'architecture MVC : 1
2.1.1 Modèle (Model) : 1
2.1.2 Vue (View) : 1
2.1.3 Contrôleur (Controller) : 2
2.2 Patron de conception Strategy : 2
2.3 Patron de conception Singleton : 3
2.4 Patron de conception Factory : 3
2.5 Patron de conception Decorator : 4
3 Tests : 5
3.1 Tests automatiques : 5
3.2 Tests manuels : 5
3.2.1 Vérification des types de scoring et de tri : 5
3.2.2 Fonctionnement des boutons : 5
3.2.3 Tests de l'interface graphique : 5
4 Fonctionnalités supplémentaires : 6
4.1 Mode sombre : 6
4.2 Chargement d'une configuration à partir d'un fichier JSON : 6
4.3 Organisation des Messages par Propriétaire : 6
5 Ethique : 6

Table des illustrations :

Figure 1 : Interface de l'application 1
Figure 2 : Modèle MVC 2
Figure 3 : UML - pattern Strategy 2
Figure 4 : Pattern Singleton 3
Figure 5 : Patterne Factory 4
Figure 6 : Pattern Décorateur 4
Figure 7 : Mode sombre 6
Figure 8 : Message d'erreur lorsqu'on essaye de supprimer un message non propritaire 6

1 Présentation du projet :

Dans le cadre de l'Unité d'enseignement « MIF01 - Gestion de Projet et Génie Logiciel », il nous a été demandé de développer en Java une application de gestion des commentaires d'un réseau social comme X. Le but du projet étant de mettre en pratique les bonnes manières de programmation, tant dans la forme que dans le fond.

1.1 Fonctionnement de l'application :

En lançant l'application, l'interface principale de l'application est affichée (cf. figure 1). Cette interface est composée d'un menu permettant de choisir une stratégie de tri, un champ textuel pour écrire le message à publier, tagger ou mentionner un ami. Différents boutons pour publier, Bookmarker ou supprimer un message.

Pour fermer l'application, il suffit de fermer la fenêtre principale.



Figure 1 : Interface de l'application

2 Design Patterns :

L'utilisation de patron de conception et d'architecture est une partie importante du projet. Nous allons donc vous présenter nos choix.

2.1 Patron d'architecture MVC :

Le patron d'architecture MVC (Modèle-Vue-Contrôleur) permet de structurer une application en trois composants distincts, chacun ayant une responsabilité bien définie. Cela permet de séparer les préoccupations et de rendre le code plus modulable et maintenable.

2.1.1 Modèle (Model) :

- Le modèle gère les données de l'application. Il contient toute la logique métier et assure la gestion des messages, utilisateurs, et stratégies de scoring dans l'application. Le modèle ici est représenté par la classe Y, qui contient les données sur les utilisateurs, leurs messages, et les stratégies de scoring.

- Le modèle est responsable de la logique métier et de la mise à jour des données.
- Il est observable : il notifie les vues (observers) lorsque les données sont modifiées.

2.1.2 Vue (View) :

- La vue est responsable de l'interface graphique. Elle affiche les données du modèle sous une forme compréhensible et permet à l'utilisateur d'interagir avec l'application. Dans ton application, la vue est gérée par la classe JfxView.
- Elle s'abonne aux notifications du modèle (via l'interface Observer) et met à jour l'interface graphique en fonction des changements dans les données. Par exemple,

lorsqu'un utilisateur publie un message ou qu'une nouvelle stratégie de scoring est appliquée.

2.1.3 Contrôleur (Controller) :

- Le contrôleur fait le lien entre la vue et le modèle. Il récupère les actions de l'utilisateur à partir de la vue et modifie le modèle en conséquence. Il est responsable de la logique d'application qui fait évoluer les données en fonction des interactions utilisateur.
- Dans ce cas, la classe Contrôleur contient la logique qui permet d'ajouter un message, de changer la stratégie de scoring, de trier les messages, etc.

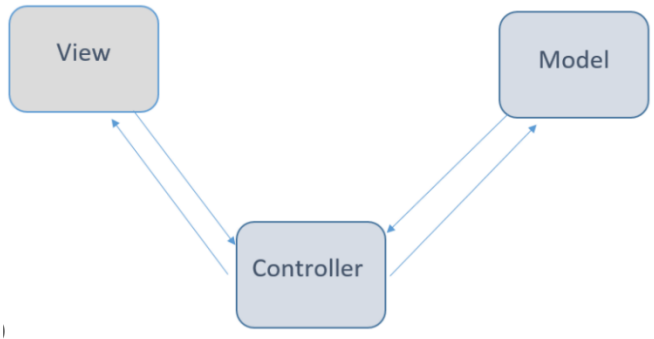


Figure 2 : Modèle MVC

Dans ce modèle pull-based, la vue interroge le modèle pour obtenir les données nécessaires à l'affichage, mais uniquement en lecture. La vue est passive et ne modifie pas directement le modèle. C'est le contrôleur qui, en réponse aux actions de l'utilisateur, met à jour le modèle.

Le contrôleur récupère les actions de l'utilisateur via la vue et met à jour le modèle en conséquence. Une fois que le modèle a été mis à jour, il notifie les vues, qui se chargent de la mise à jour de l'interface utilisateur.

2.2 Patron de conception Strategy :

Le modèle de conception Strategy est un patron de conception comportemental qui permet de définir une famille d'algorithmes, de placer chacun d'eux dans une **classe distincte** et de rendre leurs objets **interchangeables** pendant l'exécution. Ainsi, il offre un moyen très **flexible** d'appliquer différentes logiques en fonction de besoins spécifiques, mais ne viole aucun des principes de conception **SOLID**.

- Dans le cadre de notre application de microblogging, nous avons décidé d'utiliser ce modèle pour créer différentes stratégies de **scoring (BookmarkScoring, RecentMessageScoring, ForbiddenWordsScoring).** Pour cela, nous avons une **interface** générique qui définit une méthode, laquelle sera implémentée dans les **classes dérivées**. Nous avons choisi ce modèle pour les raisons suivantes :
- **Flexibilité** : Ce pattern nous permet de modifier ou d'ajouter de nouvelles règles de calcul (stratégies) sans toucher au code existant, même au moment actuel, il est tout à fait possible d'ajouter de nouvelles règles sans toucher le code existant.
- **Extensibilité** : En encapsulant chaque logique dans une classe distincte, le système est facile à étendre, notamment pour intégrer de nouvelles fonctionnalités.
- **Séparation des préoccupations** : Chaque stratégie est isolée dans sa propre classe, ce qui favorise une meilleure organisation du code et simplifie sa maintenance.

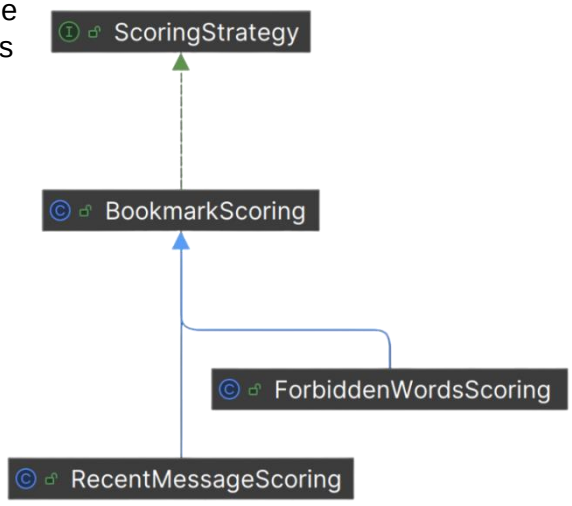


Figure 3 : UML - pattern Strategy

- **Réutilisabilité** : Les différentes stratégies peuvent être utilisées dans plusieurs contextes, ce qui réduit la duplication de code.

De plus, ce pattern respect les principes **SOLID**, notamment le principe **d'ouverture/fermeture (OCP),** qui permet de concevoir un système **extensible** sans modifier les composants existants. Ainsi, en **isolant** chaque logique, le code devient plus **maintenable et testable**, facilitant ainsi les tests unitaires et la correction des erreurs.

2.3 Patron de conception Singleton :

Le Singleton pattern est un patron conception de **création**, il garantit qu'une classe n'a **qu'une seule instance**, et fournit un point d'accès global à cette instance. Dans le contexte de notre application, la classe **Y** représente le modèle central du système de microblogging. Donc on a opté d'utiliser ce patron pour assurer qu'il n'y a **qu'une seule instance de Y**. Cela aide à maintenir un état **cohérent** à travers l'application, en évitant les problèmes liés à **plusieurs** instances, et garantissant que toutes les parties de l'application **interagissent avec les mêmes données.**

Nous avons choisi ce modèle pour plusieurs raisons :

- **Accès contrôlé à l'instance unique** : Le pattern Singleton restreint l'instanciation de la classe Y à un seul objet, assurant un accès contrôlé.
- **Point d'accès global :** Il fournit un point d'accès global à l'instance, ce qui facilite la gestion et l'accès à la ressource partagée.
- **Création & initialisation à la demande** : Cela garantit la création de l'instance uniquement lorsqu'elle est nécessaire, ce qui peut améliorer les performances, surtout lorsque la création de l'instance est coûteuse.

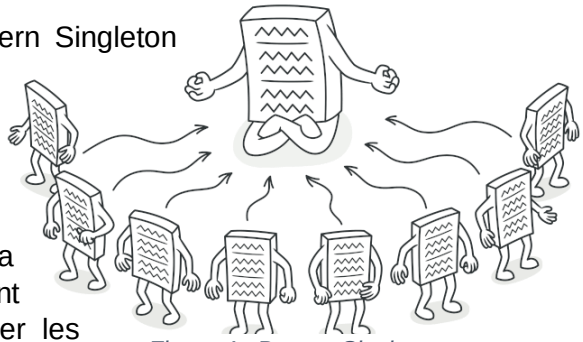


Figure 4 : Pattern Singleton

Pour implémenter le pattern Singleton, nous avons rendu le **constructeur** de la classe Y **privé** afin d'empêcher l'instanciation directe. Nous avons ensuite créé une variable d'instance **statique** pour stocker l'instance unique de la classe. Et finalement ajouté des méthodes **statiques publique**, « **getInstance »** et « **resetInstance »**, qui fournissent l'accès à cette instance en la créant si elle **n'existe** pas, et à la réinitialiser, ce qui est particulièrement utile pour les tests.

2.4 Patron de conception Factory :

Le Factory Pattern est un patron de **création** qui permet **d'encapsuler** la logique de construction d'objets complexes dans des classes dédiées (**factories**). Ce pattern **centralise et simplifie** l'instanciation d'objets, et nous 'avons choisi pour les raisons suivantes :

- **Amélioration de l'extensibilité** : En utilisant des factories pour chaque stratégie de scoring, il devient facile d'ajouter de nouvelles stratégies sans modifier le code existant.

- **Séparation des préoccupations** : La logique d'instanciation est déléguée aux factories, ce qui évite d'encombrer la vue ou le contrôleur.
- **Découplage** : La vue ne dépend pas directement des implémentations spécifiques des stratégies de scoring. Cela respecte le modèle MVC en maintenant une séparation claire entre la vue et la logique métier du modèle.

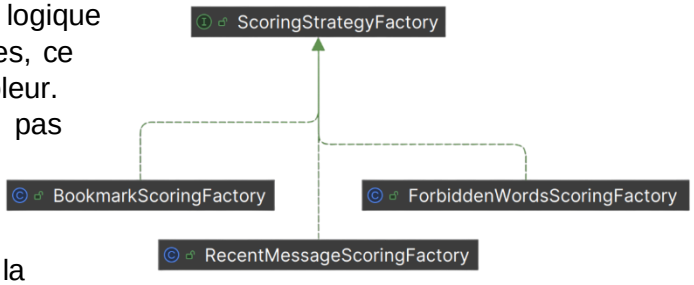


Figure 5 : Patterne Factory

- **Centralisation de la création d'objets** : Toute la logique d'instanciation est regroupée dans des classes dédiées, ce qui rend le code plus lisible et organisé, ce qui facilite sa lecture et son évolution.

En résumé, l'utilisation du **Factory Pattern** a permis d'améliorer la **flexibilité**, **l'extensibilité** et la **modularité** de notre système, tout en maintenant une **séparation stricte** entre les différentes couches de l'application.

2.5 Patron de conception Decorator :

Le pattern Decorator est un modèle **structurel** qui permet d'ajouter **dynamiquement** des responsabilités ou des fonctionnalités supplémentaires à des objets**, sans modifier** leur structure de base. Cela permet de **maintenir** les classes simples tout en offrant une **grande flexibilité pour les extensions.**

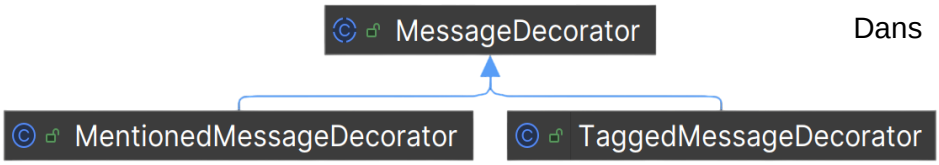


Figure 6 : Pattern Décorateur

Dans le cadre de notre application, nous avons utilisé le pattern Décorateur pour pouvoir attribuer à chaque message posté **une mention (tag)**, **un mot-dièse(hashtag)**, les deux, ou bien aucun, selon les **préférences** de l'utilisateur. Nous avons choisi ce modèle pour les raisons suivantes :

- **Flexibilité améliorée** : En encapsulant les fonctionnalités additionnelles (mentions et tags) dans des décorateurs distincts, il est possible de composer dynamiquement différents comportements, c'est-à-dire que les décorateurs peuvent être empilés dans n'importe quel ordre.
- **Respect des principes SOLID** : Le pattern respecte particulièrement le principe de responsabilité unique (Single Responsibility Principle) en séparant les fonctionnalités dans des classes spécifiques, et le principe d'ouverture/fermeture en permettant l'ajout de nouvelles fonctionnalités sans modifier les classes existantes.
- **Réutilisabilité et maintenabilité :** Les décorateurs peuvent être combinés ou réutilisés indépendamment, rendant le système plus modulaire et facile à maintenir.

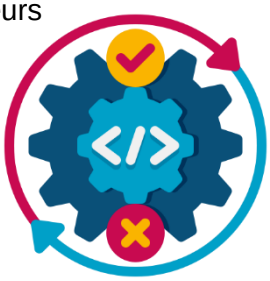
Finalement, Les mentions et les tags sont des **responsabilités supplémentaires** qui peuvent varier selon les messages. En utilisant le **Decorator** Pattern, ces fonctionnalités peuvent **être ajoutées ou retirées facilement sans affecter le message de base**. Ainsi, de nouveaux types de décorateurs peuvent être ajoutés **sans modifier les classes existantes.**

L'utilisation du **Decorator Pattern** nous a permet de gérer de manière **flexible** et **modulaire** les mentions et les tags dans les messages. Et a contribué à **améliorer la maintenabilité, la réutilisabilité et l'évolutivité** de notre application, tout en respectant les principes de conception modernes.

3 Tests :

3.1 Tests automatiques :

Concernant les tests unitaires, nous avons décidé d'effectuer les tests de chaque classe dans des fichiers séparés, ce qui permet une meilleure compréhension. Nous avons testé la majorité de nos classes et méthodes, ainsi que les cas d'erreurs possibles, pour nous assurer qu'il n'y a aucun problème dans le code. Pour les fonctions publiques, nous avons utilisé des assertions classiques avec JUnit.



3.2 Tests manuels :

Concernant les tests manuels, nous avons vérifié que les différents types de scoring et de tri sont bien pris en compte lors de la publication d'un message, et que les différents boutons fonctionnent correctement. Voici tous les éléments que nous avons testés pour nous assurer que l'interface graphique fonctionne sans erreurs :

3.2.1 Vérification des types de scoring et de tri :

- Sélection de la stratégie de scoring : nous avons vérifié que la sélection d'une stratégie de scoring dans la ComboBox met à jour correctement la stratégie de scoring de l'utilisateur et trie les messages en conséquence.
- Stratégies de scoring disponibles : nous avons testé les stratégies suivantes : Bookmarked Messages, Recent Messages, No Forbidden Words.
- Tri des messages : après la sélection d'une stratégie de scoring, nous avons vérifié que les messages sont triés correctement selon la stratégie choisie, et cela en traitant tous les cas possibles.

3.2.2 Fonctionnement des boutons :

- Bouton de publication : Nous avons testé que le bouton "Publish" permet de publier un message avec ou sans tags et mentions.
- Bouton de bookmark : Nous avons vérifié que le bouton de bookmark (★/☆) fonctionne correctement, permettant d'ajouter et de retirer un bookmark sur un message.
- Bouton de suppression : Nous avons testé que le bouton de suppression (✕) fonctionne correctement, permettant de supprimer un message si l'utilisateur en est le propriétaire.
- Bouton de changement de thème : Nous avons vérifié que le bouton de changement de thème (Dark Mode/Light Mode) applique correctement le thème sélectionné à l'interface graphique.

3.2.3 Tests de l'interface graphique :

- Affichage des utilisateurs : Nous avons vérifié que tous les utilisateurs sont affichés correctement dans l'interface, avec leurs messages respectifs.
- Affichage des messages : Nous avons testé que les messages sont affichés correctement avec leur contenu, propriétaire, date et score.
- Mise à jour en temps réel : Nous avons vérifié que l'interface se met à jour en temps réel lorsque des messages sont ajoutés, supprimés ou modifiés.

- Gestion des erreurs : Nous avons testé que les erreurs (par exemple, tentative de suppression d'un message par un utilisateur non-propriétaire) sont gérées correctement et affichent des alertes appropriées.
- Ces tests manuels nous ont permis de nous assurer que l'interface graphique fonctionne sans erreurs et répond correctement aux interactions de l'utilisateur.

4 Fonctionnalités supplémentaires :

4.1 Mode sombre :

Le mode sombre permet aux utilisateurs de passer à un thème sombre, réduisant la fatigue oculaire lors d'une utilisation prolongée. Cette fonctionnalité a été implémentée en ajoutant un bouton de bascule qui applique une feuille de style CSS différente (dark-theme.css). Les styles des composants graphiques ont été ajustés pour garantir leur cohérence et leur lisibilité dans les deux thèmes.

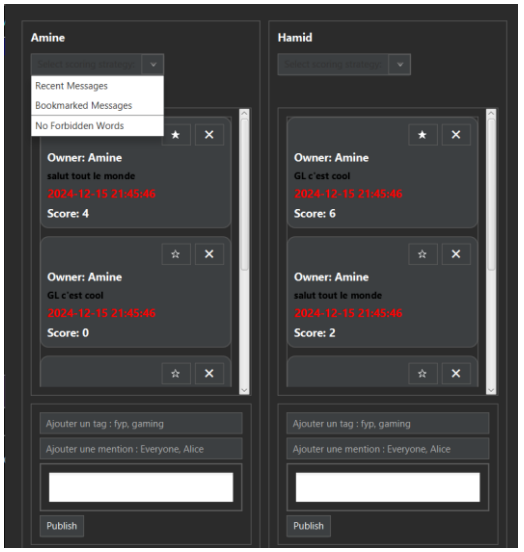


Figure 7 : Mode sombre

4.2 Chargement d'une configuration à partir d'un fichier JSON :

Cette fonctionnalité permet de charger les paramètres de configuration de l'application à partir d'un fichier JSON, offrant ainsi une flexibilité accrue aux utilisateurs pour personnaliser l'application sans modifier le code source. Les modifications incluent l'ajout d'une méthode pour lire et appliquer les configurations à partir d'un fichier JSON.

4.3 Organisation des Messages par Propriétaire :

Chaque message dans l'application est désormais associé à un propriétaire spécifique, ce qui permet de mieux organiser les messages et de savoir qui a posté quoi. Les modifications incluent l'ajout d'un champ propriétaire dans la classe Message et la mise à jour des méthodes de création et d'affichage des messages pour inclure cette information. De plus, pour renforcer la sécurité et l'intégrité des messages, seuls les propriétaires peuvent les supprimer. Cette fonctionnalité empêche les utilisateurs de supprimer les messages des autres, assurant ainsi que les messages restent intacts et que les utilisateurs ne peuvent pas altérer le contenu des autres. Les changements incluent la vérification du propriétaire avant de permettre la suppression d'un message.

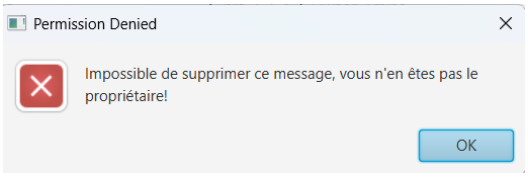


Figure 8 : Message d'erreur lorsqu'on essaye de supprimer un message non propritaire

5 Ethique :

Dans notre application nous avons implémenter plus stratégie de scoring pour proposer à l'utilisateur le contenu qu'on suppose qu'il aimera comme le font la plupart des réseaux sociaux. Cette manière de faire à plusieurs en jeux, entre autres :

La désinformation car les fake news se propagent très vite, l'influence sur l'opinion des électeurs, en proposant aux utilisateurs ce qu'ils aiment via ces algorithmes, les gens passent beaucoup de temps sur leurs téléphones ce qui engendre des problèmes

d’addictions. Ce genre d’algorithmes entraines la monétisation des données personnels et la modération des contenus entraine la violation de la liberté d’expression.

L’impact de ce genre d’outils est énorme sur la société, le fait de montrer aux utilisateurs des idéologies proches des leurs entraine des conséquences comme une étude nuance par exemple l'impact de l'éclatement des bulles de filtre sur notre capacité à nous remettre en question (www.franceinter.fr). Ces algorithmes ne favorisent pas la tolérance dans la société.

Pour remédier à ces impactes, chacun doit être conscient de l’existence de ces algos et chercher volontairement des idées contradictoires aux siennes. Il faut aussi motiver l’utilisation des moteurs de recherche qui ne font pas le tri personnalisé pour lutter contre les bulles de filtres tels que Qwant ou Ecosia. Les géants du web doivent travailler sur des mesures pour en limiter l’impact comme l’a dit Eli Pariser dans son article « Bulle de filtres ». Il faut aussi plus des sites permettant à chacun de prendre conscience de ce phénomène, comme *allsides.com.*

